COMPUTER VISION

Subject Code: 21CSE454T

ASSIGNMENT - UNIT 3

Binary Shape Analysis

Task:

Write a Python script to perform Mathematical Morphology. Read a silhouette image, perform Erosion, Dilation, Opening, and Closing, and extract the structural 8-directional Chain Code representation of the isolated binary boundary.

SUBMITTED BY

Name	**:**	Srevarshan
Register Number	**:**	RA2311026050039
Year	**:**	4th Year
Department	**:**	CSE - AIML
Section	**:**	A
Subject	**:**	Computer Vision
Subject Code	**:**	21CSE454T

1. AIM

To perform Erosion, Dilation, Opening and Closing on a silhouette image, and to find the 8-directional chain code of the object boundary.

2. ALGORITHM / PROCEDURE

A. Morphological Operations

Step 1: Read the silhouette image and convert it to a binary image.

Step 2: Add some small white dots and black holes to act as noise.

Step 3: Create a 7 x 7 elliptical structuring element.

Step 4: Erosion: a pixel stays white only if the whole structuring element fits inside the object.

Step 5: Dilation: a pixel becomes white if the structuring element touches the object.

Step 6: Opening = Erosion followed by Dilation. Closing = Dilation followed by Erosion.

Step 7: Apply Opening and then Closing to get a clean silhouette.

B. Chain Code

Step 1: Keep the largest connected object.

Step 2: Trace the outer boundary of the object pixel by pixel.

Step 3: For each move to the next boundary pixel, write the direction number (0 = East, 1 = North-East, 2 = North, ... 7 = South-East).

Step 4: Rebuild the boundary from the chain code to check that it is correct.

3. PYTHON CODE

File name: unit3_morphology_chaincode.py

```
"""
Unit 3 - Binary Shape Analysis
Mathematical morphology (Erosion, Dilation, Opening, Closing) on a silhouette
image and 8-directional Freeman chain code of the isolated object boundary.
"""
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
from numpy.lib.stride_tricks import sliding_window_view
from skimage import data

OUT = os.path.join(os.path.dirname(os.path.abspath(__file__)), "outputs")
os.makedirs(OUT, exist_ok=True)
rng = np.random.default_rng(7)

# ---------------- 1. Read silhouette & binarise ----------------
gray = (~data.horse()).astype(np.uint8) * 255          # horse = white, background = black
cv2.imwrite(os.path.join(OUT, "input_silhouette.png"), gray)

# Simulate a realistic noisy mask: white specks outside + black holes inside
noisy = gray.copy()
for val in (255, 0):
    ys, xs = rng.integers(0, gray.shape[0], 150), rng.integers(0, gray.shape[1], 150)
    for y, x in zip(ys, xs):
        cv2.circle(noisy, (int(x), int(y)), int(rng.integers(1, 3)), val, -1)
_, binary = cv2.threshold(noisy, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
cv2.imwrite(os.path.join(OUT, "input_noisy_binary.png"), binary)


# ---------------- 2. Morphological operators (custom) ----------------
SE = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))   # structuring element


def erode(img, se=SE):
    p = se.shape[0] // 2
    win = sliding_window_view(np.pad(img, p, constant_values=255), se.shape)
    return np.where(se == 1, win, 255).min(axis=(2, 3)).astype(np.uint8)   # fits entirely


def dilate(img, se=SE):
    p = se.shape[0] // 2
    win = sliding_window_view(np.pad(img, p, constant_values=0), se.shape)
    return np.where(se == 1, win, 0).max(axis=(2, 3)).astype(np.uint8)     # hits at least
        once


def opening(img):  return dilate(erode(img))      # removes small white specks
def closing(img):  return erode(dilate(img))      # fills small black holes


ops = {"Erosion": erode(binary), "Dilation": dilate(binary),
       "Opening": opening(binary), "Closing": closing(binary)}
clean = closing(opening(binary))                  # open then close -> clean silhouette

print("Custom vs OpenCV (pixels that differ):",
      int((ops["Erosion"] != cv2.erode(binary, SE, borderValue=255)).sum()),
      int((ops["Dilation"] != cv2.dilate(binary, SE, borderValue=0)).sum()))
print(f"\n{'Image':<22}{'White px':>10}{'Components':>12}{'Holes':>8}")


def stats(img):
    n_obj = cv2.connectedComponents(img)[0] - 1
    n_holes = cv2.connectedComponents(255 - img)[0] - 2       # minus outer background
    return int((img > 0).sum()), n_obj, max(n_holes, 0)
```

```
for name, im in [("Original silhouette", gray), ("Noisy binary input", binary), *ops.items(),
                 ("Open + Close (clean)", clean)]:
    w, c, h = stats(im)
    print(f"{name:<22}{w:>10}{c:>12}{h:>8}")

# ---------------- 3. Isolate object & extract boundary ----------------
n, labels, st, _ = cv2.connectedComponentsWithStats(clean)
largest = 1 + np.argmax(st[1:, cv2.CC_STAT_AREA])
obj = np.where(labels == largest, 255, 0).astype(np.uint8)
contours, _ = cv2.findContours(obj, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
boundary = contours[0][:, 0, :]                          # (x, y) points, 8-connected

# ---------------- 4. Freeman 8-directional chain code ----------------
#   3 2 1
#   4 . 0        (y axis points down in images)
#   5 6 7
DIRS = {(1, 0): 0, (1, -1): 1, (0, -1): 2, (-1, -1): 3,
        (-1, 0): 4, (-1, 1): 5, (0, 1): 6, (1, 1): 7}
pts = np.vstack([boundary, boundary[:1]])              # close the loop
chain = [DIRS[(int(dx), int(dy))] for dx, dy in np.diff(pts, axis=0)]
first_diff = [(chain[i] - chain[i - 1]) % 8 for i in range(len(chain))]   # rotation invariant

# Verify: rebuild the boundary from start point + chain code
inv = {v: k for k, v in DIRS.items()}
rebuilt = [tuple(boundary[0])]
for c in chain[:-1]:
    rebuilt.append((rebuilt[-1][0] + inv[c][0], rebuilt[-1][1] + inv[c][1]))
ok = np.array_equal(np.array(rebuilt), boundary)

perim = sum(1 if c % 2 == 0 else np.sqrt(2) for c in chain)
print(f"\nObject area            : {int((obj > 0).sum())} px")
print(f"Boundary start (x, y)  : {tuple(int(v) for v in boundary[0])}")
print(f"Chain code length      : {len(chain)}")
print(f"Perimeter from code    : {perim:.1f} px  (cv2.arcLength = {cv2.arcLength(contours[0],
    True):.1f})")
print(f"Reconstruction correct : {ok}")
print(f"Direction histogram    : {dict(zip(range(8), np.bincount(chain,
    minlength=8).tolist()))}")
print(f"Chain code (first 64)  : {''.join(map(str, chain[:64]))}")
print(f"First difference (64)  : {''.join(map(str, first_diff[:64]))}")
with open(os.path.join(OUT, "chain_code.txt"), "w") as f:
    f.write(f"Start point (x,y): {tuple(int(v) for v in boundary[0])}\nLength:
        {len(chain)}\n\n")
    f.write("Chain code:\n" + "".join(map(str, chain)) + "\n\nFirst difference:\n" +
        "".join(map(str, first_diff)))

# ---------------- 5. Figures ----------------
fig, ax = plt.subplots(2, 3, figsize=(15, 9.5))
for a, (t, im) in zip(ax.ravel(), [("Noisy binary input", binary), *ops.items(), ("Opening ->
    Closing (clean)", clean)]):
    a.imshow(im, cmap="gray"); a.set_title(t, fontsize=13); a.axis("off")
plt.suptitle("Morphological operations (7x7 elliptical structuring element)", fontsize=14,
    fontweight="bold")
plt.tight_layout(); plt.savefig(os.path.join(OUT, "fig1_morphology.png"), dpi=110);
    plt.close()

# Zoom to show effect on noise and fine parts
y0, x0, s = 0, 260, 140
fig, ax = plt.subplots(1, 5, figsize=(16, 3.8))
for a, (t, im) in zip(ax, [("Input", binary), *ops.items()]):
    a.imshow(im[y0:y0 + s, x0:x0 + s], cmap="gray", interpolation="nearest"); a.set_title(t);
        a.axis("off")
plt.suptitle("Zoomed region (head)", fontweight="bold")
plt.tight_layout(); plt.savefig(os.path.join(OUT, "fig2_zoom.png"), dpi=110); plt.close()

fig, ax = plt.subplots(1, 3, figsize=(17, 5.6), gridspec_kw={"width_ratios": [1.3, 1.3, 1]})
vis = cv2.cvtColor(obj // 3, cv2.COLOR_GRAY2RGB)
cv2.drawContours(vis, contours, -1, (0, 255, 0), 2)
```

```
cv2.circle(vis, tuple(int(v) for v in boundary[0]), 7, (255, 0, 0), -1)
ax[0].imshow(vis); ax[0].set_title("Isolated object + boundary (start = red dot)")
cmap = plt.get_cmap("tab10")
ax[1].scatter(boundary[:, 0], boundary[:, 1], c=[cmap(c) for c in chain], s=4)
ax[1].invert_yaxis(); ax[1].set_aspect("equal"); ax[1].set_title("Boundary coloured by chain-
    code direction")
for d in range(8):
    ax[1].scatter([], [], color=cmap(d), label=str(d))
ax[1].legend(title="Dir", ncol=8, fontsize=8, loc="upper center", bbox_to_anchor=(0.5, 0.02))
ax[2].bar(range(8), np.bincount(chain, minlength=8), color=[cmap(d) for d in range(8)])
ax[2].set_xticks(range(8)); ax[2].set_xlabel("Direction code"); ax[2].set_ylabel("Count")
ax[2].set_title("Direction histogram")
for a in ax[:2]: a.axis("off")
plt.tight_layout(); plt.savefig(os.path.join(OUT, "fig3_chaincode.png"), dpi=110); plt.close()

# Freeman direction reference diagram
fig, a = plt.subplots(figsize=(3, 3))
for (dx, dy), c in DIRS.items():
    a.arrow(0, 0, dx * 0.7, -dy * 0.7, head_width=0.12, color=cmap(c),
        length_includes_head=True)
    a.text(dx * 0.95, -dy * 0.95, str(c), ha="center", va="center", fontsize=14,
        fontweight="bold")
a.set_xlim(-1.3, 1.3); a.set_ylim(-1.3, 1.3); a.axis("off"); a.set_title("Freeman 8-direction
    code")
plt.tight_layout(); plt.savefig(os.path.join(OUT, "fig0_directions.png"), dpi=110);
    plt.close()
print("\nAll figures saved in", OUT)
```

4. INPUT IMAGE

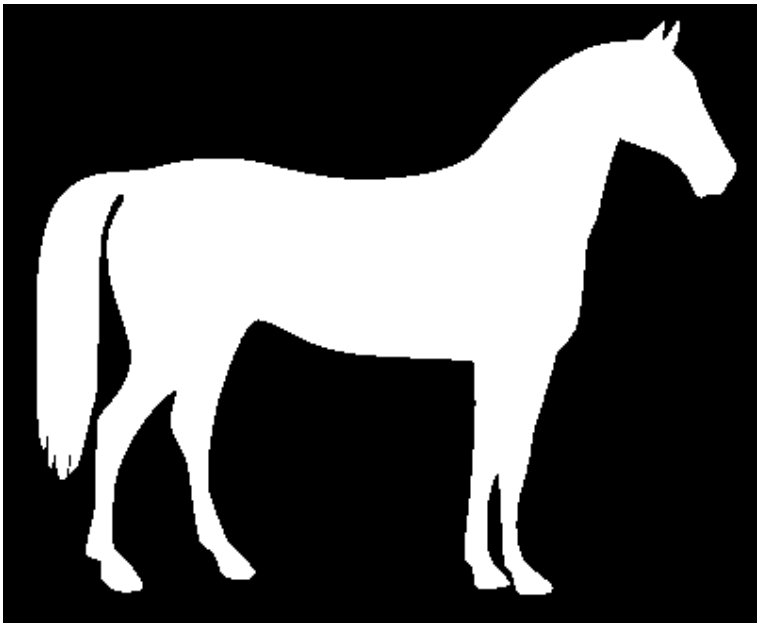


Figure 1: Input silhouette image (horse). Source: scikit-image sample data

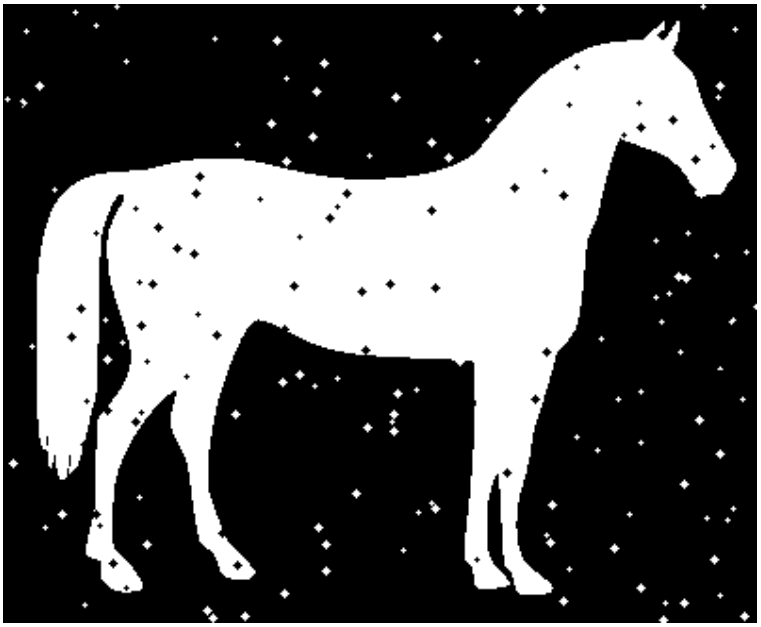


Figure 2: Binary image with added noise

5. OUTPUT SCREENSHOTS

```
Terminal

$ python unit3_morphology_chaincode.py
Custom vs OpenCV (pixels that differ): 0 0

Image                  White px  Components   Holes
Original silhouette       43412           1       1
Noisy binary input        43795         102      50
Erosion                   33835           6      24
Dilation                  56008          79       3
Opening                   42452           1      34
Closing                   44923          91       2
Open + Close (clean)      43376           1       2

Object area            : 43376 px
Boundary start (x, y)  : (344, 15)
Chain code length      : 1620
Perimeter from code    : 1833.7 px  (cv2.arcLength = 1833.7)
Reconstruction correct : True
Direction histogram    : {0: 171, 1: 151, 2: 353, 3: 110, 4: 221, 5: 123, 6: 359, 7: 132}
Chain code (first 64)  : 5455454444445444444445444454445445454545454545545545554555555
First difference (64)  : 2710717000000170000000170001700170171717171717171071071007100000

All figures saved in Unit3_Binary_Shape_Analysis/outputs
```

Figure 3: Program output in the terminal

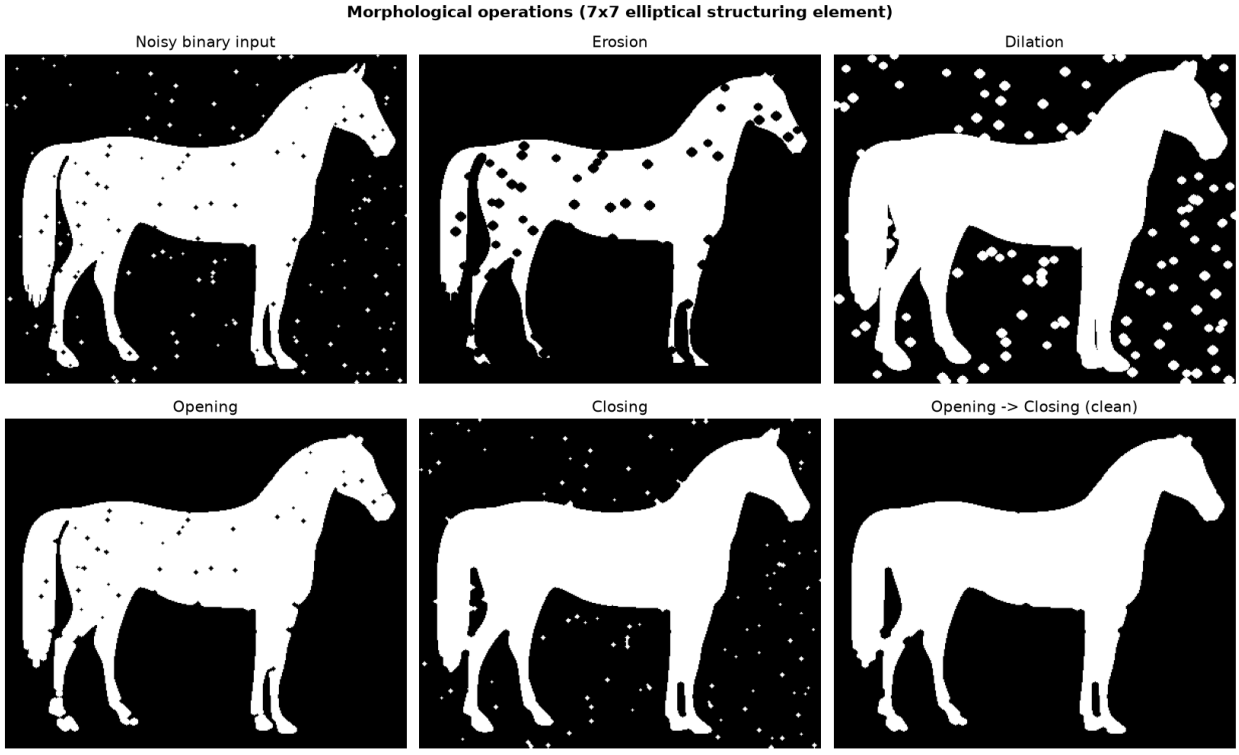


Figure 4: Erosion, Dilation, Opening and Closing

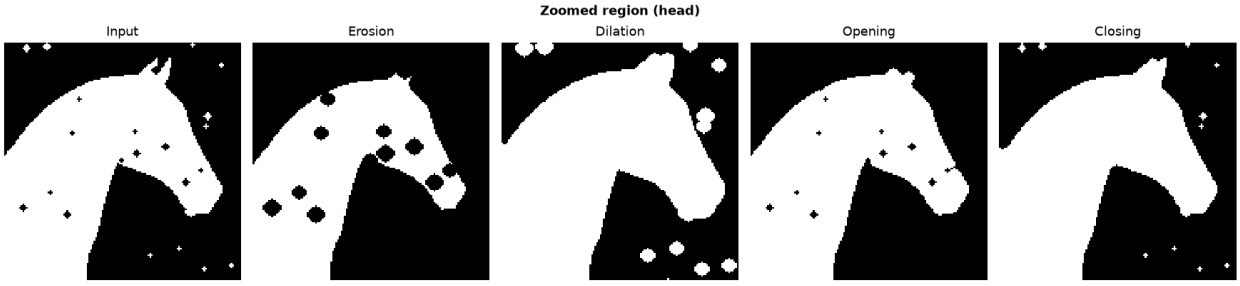


Figure 5: Zoomed view of the head region

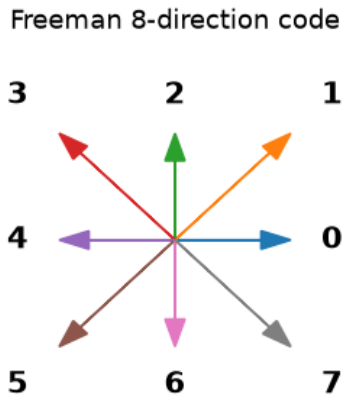


Figure 6: 8-direction chain code numbering

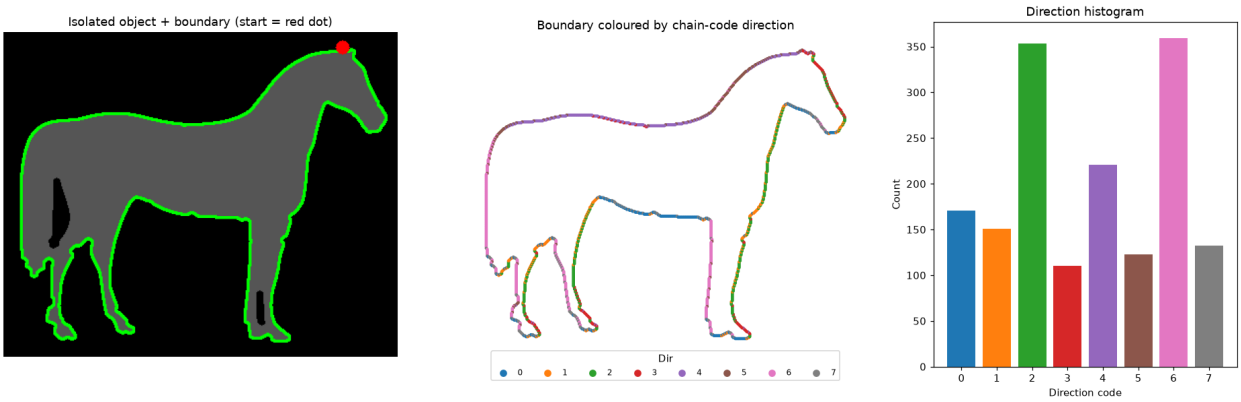


Figure 7: Object boundary and chain code direction count

6. RESULT ANALYSIS / OBSERVATIONS

Image	White pixels	Objects	Holes
Original	43,412	1	1
Noisy input	43,795	102	50
Erosion	33,835	6	24
Dilation	56,008	79	3
Opening	42,452	1	34
Closing	44,923	91	2
Opening + Closing	43,376	1	2

Chain code result	Value
Starting point (x, y)	(344, 15)
Length of chain code	1620
Perimeter	1833.7 pixels
Boundary rebuilt from code	Correct
First 40 codes	5455454444444544444444544445444544545454

- Erosion made the object smaller and removed the noise dots outside it, but the holes inside became bigger.
- Dilation made the object bigger and filled the holes, but the noise dots outside also grew.
- Opening removed all the outside noise (102 objects reduced to 1). Closing filled the holes inside (50 holes reduced to 2).
- Opening followed by Closing gave a clean shape with almost the same area as the original.
- In the chain code, directions 2 (North) and 6 (South) occur most often because of the long vertical legs.

7. CONCLUSION

Erosion and dilation have opposite effects: one shrinks the object and the other grows it. Opening removes small noise and closing fills small holes, so together they gave a clean silhouette. The 8-directional chain code described the boundary completely, and the original boundary could be rebuilt from it.